



Carnegie Mellon University
Master of
Software Engineering

Availability – tactics for monitoring and redundancy

LG Architecture Training Program

Paulo Merson



Availability

Ability of a system to mask or repair faults such that the cumulative service outage period does not exceed a required value over a specified time interval.

[Bass13]

- In other words: *the system is ready when you need it*
- Availability requires:
 - Reliability – low incidence of faults
 - Performance – system responds in a timely manner

Prevent, detect, recover from faults

- Achieving availability involves some form of fault prevention, detection, or recovery
 - Specifically, the goal of fault recovery and fault tolerance is to make the application more resilient **when a fault occurs**
- Availability tactics and technologies can be applicable to
 - the application code
 - the runtime infrastructure
- Typical tradeoffs include
 - added complexity
 - performance overhead

Availability metric

- Most common metric: $MTBF / (MTBF + MTTR)$
 - **MTBF**: mean time between failure
 - **MTTR**: mean time to repair
- Example:
 - MTBF = 198 hours; MTTR = 2 hours
 - Availability $= 198 / (198 + 2)$
 $= 0.99$
 $= 99\%$

Another way to calculate:
 $uptime / (uptime + downtime)$

*Time for scheduled
maintenance outages
does not count*



*Availability is
important for SLAs*



Availability tactics

- Detect faults
 - Monitor
 - Ping/echo
 - Heartbeat
- Recover from faults (or tolerate faults)
 - Active redundancy (hot spare)
 - Passive redundancy (warm spare)
 - Cold spare
 - Timeout
 - Retry
 - Circuit breaker



Monitor

- A centralized component oversees the state of health of different elements and resources in the system, such as:
 - Application components
 - CPU
 - Memory
 - Disk and I/O
- In addition to [detecting](#) and notifying problems, it may take action to [recover](#) from a fault (e.g., restart a failed component)
- The monitor typically uses one of two models
 - [Pull](#): monitor asks monitored elements for status (see [Ping/echo](#) tactic)
 - [Push](#): monitored elements send status to monitor (see [Heartbeat](#) tactic)

Ping/echo

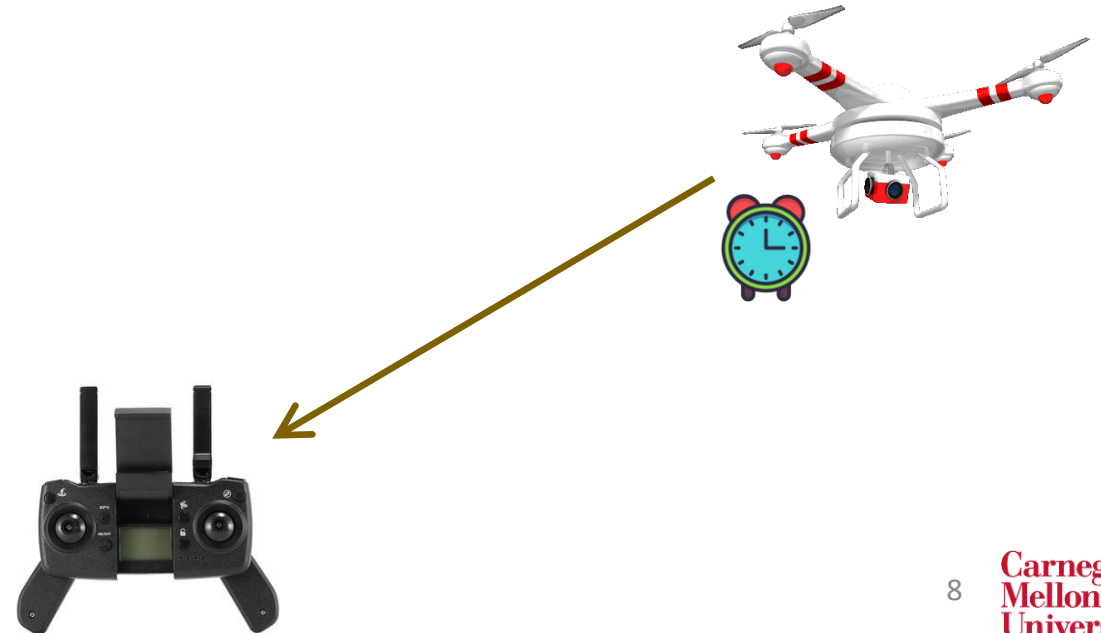
- Helps to timely **detect** faults
- Monitoring component sends “ping”; monitored component replies with “echo”
 - Can check reachability and assess roundtrip time
 - There is a predefined timeout
- Example: a Kubernetes cluster can monitor application services via liveness and readiness probes



```
containers:
  - name: my-microservice
    image: registry.acme.com/apps/my-microservice:${tag-image}
    resources:
      requests:
        cpu: 200m
        memory: 128Mi
    livenessProbe:
      httpGet:
        path: /api/v1/metrics/health
        port: app-port
      initialDelaySeconds: 60
      failureThreshold: 5
      periodSeconds: 10
      timeoutSeconds: 2
```

Heartbeat

- Also helps to timely **detect** faults
- Monitored component periodically sends a “heartbeat” message to monitoring component
- Heartbeat may piggyback on other periodic message used by the system, for example:
 - A drone periodically sends a location message to the remote navigation control system
 - The same message can carry status info (heartbeat) to the monitoring system



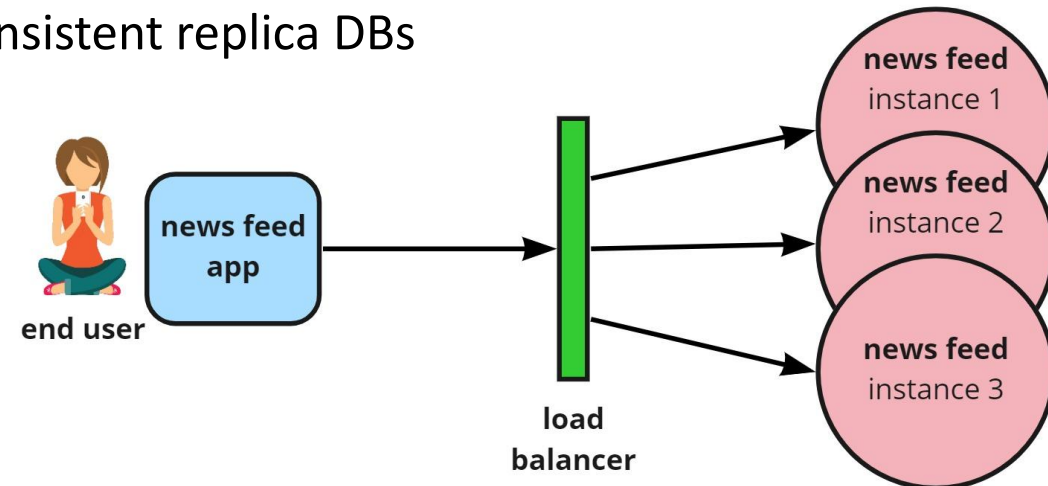
Active redundancy (hot spare)

- Helps to **recover** from a fault
- Two or more instances of the same component
 - contain the same visible state at all times
 - process the same requests (often in parallel)
- Provides the **fastest fault recovery** but has the **highest cost**
- The redundant element can be:
 - Computing nodes or components that shall process the same input in parallel
 - Data repositories that remain in sync (consistent)
 - Communication channels or network links



Passive redundancy (warm spare)

- Helps to **recover** from a fault
- Different instances can process independent requests
- Fault recovery is **slower**, but at a **lower cost** than active redundancy
- The redundant element can be:
 - Multiple instances of application services available to process incoming requests
 - Data repository with a primary DB and eventually consistent replica DBs



Cold spare

- Helps to **recover** from a fault
- The replica is only activated when a fault occurs
- State is recreated based on log records
- Provides the slowest fault recovery
- Appropriate when the primary component is highly reliable (rarely fails)
- Ex.: offsite backup and data center for **disaster recovery**



Example of technology choice around redundancy levels



Source: AWS Certified Solutions Architect (SAA-C03) training course by Chad Smith



Questions?

Paulo Merson
pmerson@cmu.edu